



Lenguajes de Programación

IT Essentials | ⌚ 19 minutos

Cómo nos comunicamos con las computadoras para darles instrucciones?

Igual que comunicarse con una persona, por medio de un **Lenguaje**.

La diferencia es que las computadores entienden lenguajes distintos a los nuestros,

Yo no puedo llegar y decirle *"hey, créame una pagina web capaz de reproducir videos"*.

Últimamente han salido muchas "inteligencias artificiales" capaces de entender lenguajes humanos como el **español** o el **ingles** - **en realidad no los entienden realmente**, solo los pasan por muchas capas de traducción hasta llegar a un lenguaje entendible por la computadora.

Es como que yo agarre un texto en latín, lo traduzca a italiano, luego a portugués y finalmente a español.

Origen	Latín
	Italiano
	Portugués
Destino	Español

No entiendo ninguno de los lenguajes anteriores, pero sí el español.

Qué lenguajes entiende la computadora?

Solo uno, se le llama `código maquina` y luce así:

```
1 0001011101010001000100011100101110101010
2 0111101111101100000110110101110111101010
3 0010001011011011001010110010101000100001
4 1100111110000110000000110001001101010110
5 001101100100011010111011011101111110010
6 0000001101101000101011111011011101001000
7 0000110011111010100000011001010100100001
8 1110001110001010100011101111010111100111
9 0110000111100101111001001001011111000011
10 1101100101101011011110101111011101101001
```

Copiar

Como vemos, consiste de únicamente de 1s y 0s, lo que también llamamos `código binario`.

Acá una buena explicación de cómo funciona.

Esto debido a que todos los componentes de una computadora funcionan con energía eléctrica.

Lo único que sabe hacer la computadora realmente, es enviar energía a sus componentes o no enviar energía (1 o 0).

Absolutamente todo lo que usted esta viendo en su pantalla es generado a partir de pulsaciones de energía, o 1s y 0s. **Billones de 1s y 0s por segundo.**

Obviamente, nadie programa con 1s y 0s, así que no te asustes.

Para programar usamos lenguajes de programación que en su mayoría son muy parecidos al ingles, pero más sencillos.

Justo como con el ejemplo del latín, los lenguajes de programación pasan por varias capas de traducción hasta llegar a código maquina para que las computadoras puedan entender las instrucciones que les estamos dando.

Niveles de lenguaje

Existe un termino llamado `lenguaje de alto nivel` o `lenguaje de bajo nivel`

Esto determina qué tan cerca se encuentre el lenguaje al código máquina en la jerarquía de traducción.

Usando el ejemplo anterior, **Latín** sería un **lenguaje de alto nivel** y **portugués** un **lenguaje de bajo nivel** (ya que es *el más cercano* al español).

Mientras más bajo sea el nivel del lenguaje, más rápido será. Pero también más complicado de entender y escribir.

Por ejemplo, veamos cómo se escribe un programa que muestre el mensaje "Hello, World!" en los siguientes lenguajes:

1. **Assembly** (el lenguaje más cercano a código máquina)
2. **C** (un lenguaje de bajo nivel)
3. **Python** (un lenguaje de alto nivel)
4. **Javascript** (otro lenguaje de alto nivel)

Assembly

```
1  org  0x100
2
3  mov  dx, msg
4  mov  ah, 9
5  int  0x21
6
7  mov  ah, 0x4c
8  int  0x21
9
10 msg  db 'Hello, World!', 0x0d, 0x0a, '$'
```

[Copiar](#)

C

```
1  #include <stdio.h>
2
3  int main() {
4      printf("Hello, World!");
5      return 0;
6  }
```

[Copiar](#)

Python

```
1 print("Hello, World!")
```

[Copiar](#)

Javascript

```
1 console.log("Hello, World!");
```

[Copiar](#)

Como vemos, lenguajes como Python o Javascript son mucho mas sencillos.

Esto se debe principalmente a que son lenguajes de alto nivel.

Pasan por muchas capas de traducción antes de llegar a código maquina.

Esto los hace mas lento que un lenguaje de medio o bajo nivel, pero mucho mas fácil de escribir y entender.

Son casi como escribir en ingles puro.

Compilación e interpretación

Existen 2 maneras en que las instrucciones de lenguaje de programación se pueden convertir a código maquina: la **compilacion** y la **interpretacion**.

Los lenguajes **compilados** deben de traducir todas sus instrucciones antes de poder ejecutarlas.

Los lenguajes **interpretados** van traduciendo instrucción por instrucción a medida de que se ejecutan.

Esto es un arma de doble filo, ya que si hay algún error en una línea muy adentro del código, no vamos a poder detectarlo hasta que el código pase por ahí.

```
1 print("Esto va bien")
2 print("Esto tambien")
3 printf("Aqui hay un typo")
```

[Copiar](#)

Esto va bien

Esto tambien

Traceback (most recent call last):

File "/Users/alekcastillo/Documents/repos/d0ap/interpretacion.py", line 3, in <module>

printf("Aqui hav un typo")

```
print('quiero un tipo')  
^^^^^
```

NameError: name 'prin' is not defined. Did you mean: 'print'?

Justo como con los lenguajes de alto y bajo nivel, los lenguajes compilados son mas rápidos pero mas complicados de escribir, mientras que los interpretados son mas lentos pero fáciles de escribir.

Typeados y dinámicos

Existe otra diferencia importante entre los lenguajes de programación: el si son **typeados** (typed) o **dinámicos**.

O de **typeado estático** o **typeado dinámico**.

La mayoría de *lenguajes compilados* son **typeados**.

La mayoría de *lenguajes interpretados* son **dinámicos**.

La diferencia entre estos es (básicamente) la necesidad de especificar el tipo de los datos con los que se trabaja.

Dinámicos

```
1 valor = 6
```

Copiar

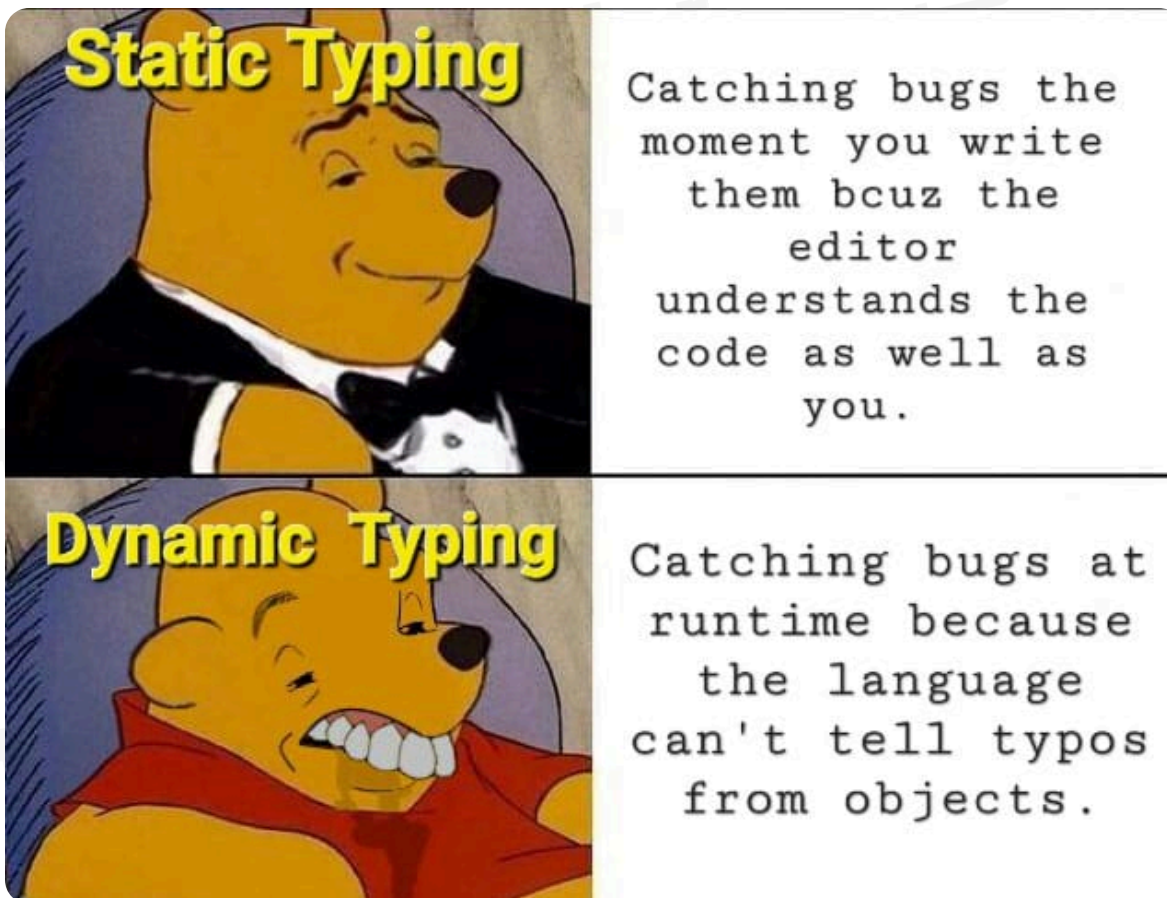
Typeados

```
1 Número valor = 6
```

Copiar

Este pequeño detalle hace que los **typeados** sean mucho más complicados para primerizos.,

Luego lo veremos más a fondo.



Aquí un meme que ya va a poder entender.

Cual es el mejor lenguaje de programación?

No existe. Si existiese, todos lo usaríamos y fin de la historia.

Todos los lenguajes de programación fueron creados con distintos propósitos.

Por ejemplo, en los 2000s, teníamos que usar `C#` si queríamos crear cualquier programa de Windows, y `Objective-C` para programas de Mac.

Si queríamos algo multiplataforma, debíamos usar `Java`.

Estas cosas aún suceden hasta cierto punto. Sin embargo, ahora la mayoría de lenguajes tienen versiones multiplataforma.

Además, la programación web nos permite crear aplicaciones que funcionen en cualquier dispositivo.

Una pagina web se va a ver 99% igual en la mayoría de navegadores.

Así mismo, el hueco entre los usos de los distintos lenguajes se ha ido cerrando.

Ahora la mayoría de lenguajes pueden ser usados para la mayoría de tareas.

La principal excepción es el frontend web - para este siempre debemos usar Javascript.

Hay varios factores que considerar a la hora de seleccionar un lenguaje para un proyecto.

Por ejemplo, si necesitamos crear código que se comunique de manera más cercana al hardware de la computadora, lo mejor sería usar un lenguaje de bajo nivel.

Por otro lado, si necesitamos escribir código fácil de leer y entender para un sistema trabajado por varias personas y equipos, lo mejor sería usar un lenguaje de alto nivel.

Los que veremos

Durante la mayoría del programa, vamos a estar trabajando con Python y Javascript.

Esto debido a que son los 2 lenguajes más demandados en la industria actual según la última encuesta de Stack Overflow:

Programming, scripting, and markup languages



2023 continues JavaScript's streak as its eleventh year in a row as the most commonly-used programming language. Python has overtaken SQL as the third most commonly-used language, but placing first for those who are not professional developers or learning to code (Other Coders).

A few technologies moved up a spot this year (Bash/Shell, C, Ruby, Perl, and Erlang) with two moving up two spots (Elixir and Lisp). The big mover, gaining seven spots since 2022 was Lua, an embeddable scripting language.

Professional developers top three technologies are the same as last year—JavaScript, HTML/CSS, and SQL.

But it's a different picture for those learning to code. HTML/CSS and JavaScript are almost tied as the most popular languages for people learning to code. Student developers use Python more than SQL (59% vs. 37%), while professional developers report using SQL more than Python (52% vs 45%).

Compared to Professional Developers, those learning to code are more likely to report using Java (37% vs 31%), C++ (32% vs 20%), and C (32% vs 17%).

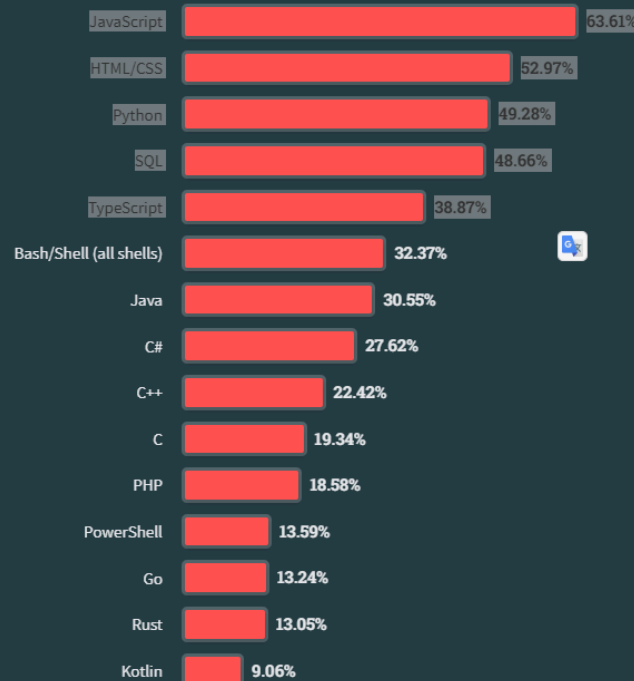
All Respondents

Professional Developers

Learning to Code

87,585 responses

Other Coders



El saber ambos da una muy buena ventaja competitiva.